

```

---
title: "algoritmos"
output:
  html_document: default
  word_document: default
  pdf_document: default
author: "Leslie Garcia"
date: "2026-04-01"
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

```

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

Primeros comandos en R

El lenguaje R es un entorno de programación ampliamente utilizado en el campo de la estadística, el análisis de datos y la ciencia de datos. Fue desarrollado con el objetivo de facilitar el procesamiento, la manipulación y la visualización de grandes volúmenes de información de manera eficiente. Además, R es un lenguaje muy parecido a MathLab y trabaja con variables que en general suelen ser matrices.

Una de las principales características de R es su capacidad para realizar análisis estadísticos avanzados, incluyendo modelos de regresión, pruebas de hipótesis y técnicas de aprendizaje automático. Además, cuenta con una gran variedad de paquetes y librerías que amplían sus funcionalidades, permitiendo abordar problemas complejos en distintas áreas como la economía, la ingeniería, la biología y las ciencias sociales.

R también se destaca por su potente sistema de visualización, que permite generar gráficos de alta calidad para representar datos de forma clara e intuitiva. Gracias a su carácter de software libre y su activa comunidad de usuarios, se ha convertido en una herramienta fundamental para investigadores, estudiantes y profesionales.

En la actualidad, el lenguaje R es una de las herramientas más importantes para el análisis de datos, siendo ampliamente utilizado en entornos académicos y profesionales.

```

```{r}
plot(pressure,main="Presión del gas ideal",ylab="Presión
[hPa]",xlab="Temperatura [K]",type="b", col="green")
```

```

```

```{r}
A<-38
A<-40
B<-60
C<-B-A
C
```

```

```

```{r}
y=40
alpha<-30
```

```

```

```{r cars}
summary(cars)
```

```

Uso de Data Set internas de R

Conclusión

El uso de R en **Posit Cloud** simplifica enormemente el acceso al análisis de datos, ya que elimina la necesidad de instalación y configuración local. Esto lo convierte en una herramienta ideal para estudiantes y trabajos colaborativos, permitiendo comenzar a programar de forma inmediata desde cualquier dispositivo.

Sin embargo, esta comodidad viene acompañada de ciertas limitaciones, como la dependencia de una conexión a internet y restricciones en los recursos computacionales. Por lo tanto, **Posit Cloud** es excelente para aprendizaje, prototipado y proyectos medianos, mientras que para tareas más pesadas o profesionales puede ser conveniente trabajar en un entorno local.

Tabla comparativa: Posit Cloud vs R local

| Característica | Posit Cloud | R local |
|-----------------------|---------------------------------|-----------------------------|
| Instalación | No requiere | Requiere instalación |
| Acceso | Desde cualquier navegador | Solo en el dispositivo |
| Configuración inicial | Muy simple | Puede ser compleja |
| Uso de recursos | Limitado (según plan) | Depende del hardware propio |
| Conexión a internet | Obligatoria | No necesaria |
| Velocidad | Depende de internet | Generalmente más rápida |
| Trabajo colaborativo | Muy fácil (compartir proyectos) | Más difícil |
| Personalización | Limitada | Alta |
| Uso educativo | Excelente | Bueno |
| Proyectos grandes | Limitado | Ideal |

Including Plots

You can also embed plots, for example:

```
```{r pressure, echo=FALSE}
plot(pressure)
```
```

Data Set en R

¿Qué es un dataset?

Es un conjunto de datos organizado (tabla con filas y columnas) que se usa para análisis estadístico.

Tipos de datasets en R:

Internos → vienen incluidos en R o paquetes

Ej: `iris`, `mtcars`

Externos → archivos que importás

Ej: CSV, Excel, TXT

Datasets internos

- Ya están disponibles sin descargar
- Se usan para aprender y practicar

Comandos básicos:

```
```{r}
data() # lista todos los datasets
data(iris) # carga uno
```
```

Datasets externos

- Se cargan desde archivos

Ejemplos:

```
```r
read.csv("archivo.csv")
read.table("archivo.txt")
```
```

Exploración de datos

```
```r
```

```
head(datos) # primeras filas
summary(datos) # resumen estadístico
str(datos) # estructura
```

```

```
#### Idea clave
```

Los datasets son la base del análisis en R:

```
**sin datos, no hay análisis.**
```

Permiten aplicar estadísticas, visualizar información y tomar decisiones.

```
##Funciones Estadísticas
```

```
```{r}
z1<-rnorm(351,22,5)
z1
```
```

```
```{r}
plot(z1)
```
```

```
```{r}
hist(z1, main="Histograma de edades", breaks=50)
```
```

```
```{r}
density(z1)
```
```

```
```{r}
plot(density(z1),type="p",col="lightblue")
lines(density(z1),col="green")
```
```

```
## Normal {stats} R Documentation
```

```
### The Normal Distribution
```

```
#### Description
```

Density, distribution function, quantile function and random generation for the normal distribution with mean equal to mean and standard deviation equal to sd.

Usage

```
dnorm(x, mean = 0, sd = 1, log = FALSE)
pnorm(q, mean = 0, sd = 1, lower.tail = TRUE, log.p = FALSE)
qnorm(p, mean = 0, sd = 1, lower.tail = TRUE, log.p = FALSE)
rnorm(n, mean = 0, sd = 1)
```

Arguments

x, q vector of quantiles.

p vector of probabilities.

n number of observations. If length(n) > 1, the length is taken to be the number required.

```
```{r}
plot(z1)
w1<-length(z1)
w1
x1<-(500:850)
x1
plot(x1, z1, col="purple")
lines(x1, z1, type="l", col="orange")
```
```

```
## **CLASE 8 DE ABRIL**
```

****Consigna 1:**** Las ultimas 3 cifras de mi DNI son 126, consigna crear una variable que tenga ese número.

```
```{r}
DNI=126
```
```

****consigna 2:**** Crear un vector que tenga todos los números desde el 1 hasta el 126 enteros.

```
```{r}
for(i in 1:126)
 print(1:i)
```
```

```
```{r}
secuenciaDNI <- c()
for(i in 1:232){
 secuenciaDNI <- c(secuenciaDNI, i)
}
```
```

****Consigna 3:**** Calcula la suma de todos los valores del vector secuenciaDNI con R

```
```{r}
total<-0
valorfinal<-length(secuenciaDNI)
for (i in 1:valorfinal){
total<-total+secuenciaDNI[i]
}
print(total)
```
```

****Consigna 4:**** Calcula la suma de todos los valores del vector secuenciaDNI con python.

```
```{python}
secuenciaDNI = range(1, 233) # incluye 232
total = sum(secuenciaDNI)
print(total)
```
```

****Consigna 5:**** Calcula la suma de todos los valores del vector secuenciaDNI con Julia.No se programó para no ocupar memoria

****Consigna 6:**** Determinar CPU time used.

```
```{r}
require(stats)
system.time(for(i in 1:100) mad(runif(1000)))
```
```

```
```{r}
require(stats)
system.time(for(i in 1:length(secuenciaDNI)){})
total<-0
valorfinal<-length(secuenciaDNI)
for (i in 1:valorfinal){
total<-total+secuenciaDNI[i]
}
print(total)
```
```

```
```{r}
sleep_for_a_minute <- function() { Sys.sleep(14) }
start_time <- Sys.time()
sleep_for_a_minute()
end_time <- Sys.time()
end_time- start_time
```
```

****Consigna 7:**** Comparación de performance con system.time().

```
```{r}
system.time({
```

```

A <- numeric(50000)
for (i in 1:50000) {
 A[i] <- i * 2
}
})
system.time({
 B <- seq(from = 2, to = 100000, by = 2)
})
``

```

### ## Biblioteca tictoc

Esto de usar una biblioteca es llamar u cargar una procedimientos que generará comando nuevos en R. Como ya fue comentado, cargar una biblioteca implica ejecutar el comando `install.packages()` o usar en r-studio el menú de Herramientas y Luego Instalar paquetes. Las funciones `tic` y `toc` son de la misma biblioteca de Octave/Matlab y se usan de la misma manera para la evaluación comparativa que el tiempo de sistema recién demostrado. Sin embargo, `tictoc` agrega mucha más comodidad al usuario y armonía al conjunto.

```

```{r}
library(tictoc)
tic("sleeping")
A<-20
print("dormire una siestita...")
## [1] "dormire una siestita..."
Sys.sleep(2)
print("...suena el despertador")
## [1] "...suena el despertador"
toc()
```

```

### ## Biblioteca rbenchmark

La documentación de la función `benchmark` del paquete `rbenchmark` R lo describe como un simple contenedor alrededor de `system.time`. Sin embargo, agrega mucha conveniencia en comparación con las llamadas simples a `system.time`. Por ejemplo, requiere solo una llamada de referencia para cronometrar múltiples repeticiones de múltiples expresiones. Además, los resultados devueltos se organizan convenientemente en un marco de datos.

### ## Implementación de una serie Fibonachi o Fibonacci

En matemáticas, la sucesión o serie de Fibonacci es la siguiente sucesión infinita de números naturales: 0, 1, 1, 2, 3, 5, 8 ... 89, 144, 233 ... La sucesión comienza con los números 0 y 1, 2 a partir de estos, cada término es la suma de los dos anteriores, es la relación de recurrencia que la define. A los elementos de esta sucesión se les llama números de Fibonacci. Esta sucesión fue descrita en Europa por Leonardo de Pisa, matemático italiano del siglo XIII también conocido como Fibonacci. Tiene numerosas aplicaciones en ciencias de la computación, matemática y teoría de juegos. Original de la Biblioteca Universidad de Florencia. Liber Abachi- Autor Fibonacci

**\*\*Consigna 8:\*\*** ¿Cuántas iteraciones se necesitan para generar un número de la serie mayor que 20 ? Además mostrar la serie de Fibonacci hasta el número obtenido anteriormente.

```

```{r}
# Inicialización
a <- 0
b <- 1
serie <- c(a, b) # guardamos la serie
contador <- 2 # ya tenemos 2 términos
# Generar hasta superar 20
while (b <= 20) {
  siguiente <- a + b
  a <- b
  b <- siguiente

  serie <- c(serie, b) # agregamos a la serie
  contador <- contador + 1
}

```

```
# Resultados
cat("Cantidad de iteraciones:", contador, "\n")
cat("Primer número mayor a 20:", b, "\n")
cat("Serie de Fibonacci hasta ese número:\n")
print(serie)
```

```

**\*\*Consigna 9:\*\*** Compara la performance de ordenación del método burbuja vs el método sort de R. Usar método microbenchmark para una muestra de tamaño 20.000

```
```{r}
# Instalar paquete
if (!require(microbenchmark)) {
  install.packages("microbenchmark")
  library(microbenchmark)
}
# Generar muestra
set.seed(123)
n <- 20000
vector <- sample(1:100000, n, replace = TRUE)
#Implementación de ordenamiento burbuja
bubble_sort <- function(x) {
  n <- length(x)
  for (i in 1:(n-1)) {
    for (j in 1:(n-i)) {
      if (x[j] > x[j+1]) {
        temp <- x[j]
        x[j] <- x[j+1]
        x[j+1] <- temp
      }
    }
  }
  return(x)
}
#Benchmark (OJO: burbuja es muy lento)
resultado <- microbenchmark(
  burbuja = bubble_sort(vector),
  sort_R = sort(vector),
  times = 5
)
print(resultado)
```

```

## ## La penitencia de Newton

Comentan algunos historiadores que Newton tenía en la escuela primaria un profesor muy exigente que reclamaba constantemente la atención de los alumnos a lo que el estaba enseñando. Tan pronto alguien cometía un acto impropio lo castigaba con una penitencia que consistía en traer para el día siguiente el resultado de la suma desde el número 1 hasta cierto número aleatoriamente elegido. Por supuesto esta tarea demandaría quedarse toda la noche haciendo las sumas parciales, tarea que el profesor ya había realizado previamente y que tenía registrado en un voluminoso cuaderno que (al estilo tabla de logaritmos) cargaba religiosamente en su portafolios de cuero.

**\*\*Consigna 10:\*\*** Desarrolla dos algoritmos que hagan este trabajo, por ejemplo para sumar desde 1 hasta  $1 \times 10^5$  y verifica cuál de los dos es más eficiente.

```
```{r}
#Algoritmo 1: suma con for (iterativo)
n <- 1010
suma1 <- 0
for (i in 1:n){
  suma1 <- suma1 + i
}
suma1
#Complejidad:O(n)(suma uno por uno)
#Algoritmo 2: fórmula matemática (Gauss)
n <- 1010

```

```

suma2 <- n * (n + 1) / 2
suma2
#Complejidad:O(1)(una sola operación)
#Comparación de eficiencia:medir tiempos con system.time():
# Método 1
system.time({
  suma1 <- 0
  for (i in 1:1010){
    suma1 <- suma1 + i
  }
})
# Método 2
system.time({
  suma2 <- 1010 * (1010 + 1) / 2
})
````

```

Se desarrollaron dos algoritmos para sumar desde 1 hasta  $1 \times 1010$ . El método iterativo presenta complejidad lineal, lo que lo hace ineficiente para valores grandes como 10.000.000.000. En cambio, el uso de la fórmula cerrada reduce la complejidad a constante, permitiendo obtener el resultado de forma inmediata. Por lo tanto, el segundo algoritmo es significativamente más eficiente.

### ##Matriz Inversa y Gráficos Violín

En este apartado se realiza una comparación del rendimiento computacional de distintos métodos para estimar los coeficientes de un modelo de regresión lineal en R. Para ello, se emplea la librería `microbenchmark`, la cual permite medir con precisión los tiempos de ejecución de diferentes algoritmos a través de múltiples repeticiones.

Se genera un conjunto de datos simulado utilizando funciones de generación aleatoria, compuesto por una matriz de variables explicativas  $X$  de dimensión  $10000 \times 100$  y un vector respuesta  $y$ , construido como una combinación lineal de  $X$  con un término de error. Esto permite trabajar en un contexto controlado pero representativo de problemas de gran escala.

A continuación, se implementan tres enfoques para el cálculo de los coeficientes: el método clásico mediante la función `lm()`, el cálculo explícito usando la inversa de la matriz y la resolución directa del sistema lineal equivalente mediante la función `solve()`. Para asegurar la validez de la comparación, se define una función que verifica que los coeficientes obtenidos por los distintos métodos sean numéricamente equivalentes dentro de una tolerancia establecida.

Finalmente, se visualizan los resultados utilizando la librería `ggplot2`, a través de la función `autoplot()`, que permite representar gráficamente la distribución de los tiempos de ejecución de cada método, facilitando su análisis comparativo.

```

````{r}
library(microbenchmark)
set.seed(2017)
n <- 10000
p <- 100
X <- matrix(rnorm(n * p), n, p)
y <- X %%% rnorm(p) + rnorm(100)
check_for_equal_coefs <- function(values) {
  tol <- 1e-12
  max_error <- max(
    c(
      abs(values[[1]] - values[[2]]),
      abs(values[[2]] - values[[3]]),
      abs(values[[1]] - values[[3]])
    )
  )
  max_error < tol
}
mbm <- microbenchmark(

```

```

"lm" = {
  b <- lm(y ~ X + 0)$coef
},
"pseudoinverse" = {
  b <- solve(t(X) %*% X) %*% t(X) %*% y
},
"linear system" = {
  b <- solve(t(X) %*% X, t(X) %*% y)
},
check = check_for_equal_coefs
)
mbm
library(ggplot2)
autoplot(mbm)
```

```

La línea: `autoplot(mbm)` genera automáticamente un gráfico a partir del objeto de `microbenchmark`, y por defecto ese gráfico es un `boxplot` (diagrama de cajas), que muestra: mediana, cuartiles, dispersión de los tiempos. No es un gráfico de violín.

Entonces, este gráfico que sigue es mucho mejor que solo `boxplot`. El violín deja ver si hay: colas largas, multimodalidad y variabilidad real.

```

```{r}
library(ggplot2)
autoplot(mbm) +
  geom_violin(fill = "darkblue", alpha = 0.4, color = NA) + # distribución
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.7) + # resumen
  stat_summary(fun = mean, geom = "point",
    shape = 23, size = 3, fill = "pink") + # media
  labs(
    title = "Comparación de tiempos de ejecución",
    x = "Método",
    y = "Tiempo (ms)"
  ) +
  theme_minimal() +
  theme(
    legend.position = "none",
    plot.title = element_text(hjust = 0.5, face = "bold")
  )
```

```